State and Cost Limits of Reference-Preserving Speculative Decoding

Anonymous Authors[1]

Abstract

Speculative decoding must both reproduce the chosen reference and repay its drafting and verification costs. We examine these requirements together in quantized consumer-system deployments. Natural state crosses on CUDA and Vulkan reveal current-path effects, retained-state effects, and local interactions despite equal committed token prefixes. On RTX, a finite attention-layout constraint applied to target prefill and continuation preserves the full reference output on 24 previously unused project inputs. Generation speed is 1.170× against ordinary decoding and 1.169× against matched draft-free execution. Resident-request speed is 1.742× against ordinary decoding, but only 1.035× against the matched path: that path already provides 1.683× without speculation. Wider drafting at fixed microbatch capacity remains qualified while becoming slower, and a separate B570 endpoint repair loses to both draft-free comparators. Historical M4 and 358H panels provide further cost boundaries under their own contracts. The contribution is a joint empirical account of numerical qualification and amortization: a positive RTX operating point, causal counterexamples, and measured limits on apparently promising repairs. The recipes and causal evidence remain deployment-specific.

1 Introduction

Speculative decoding reduces sequential target-model work by drafting candidate tokens and verifying them in parallel (Leviathan et al., 2023). In a floating-point deployment, two questions determine whether that transformation is useful. Does it reproduce the selected reference output? Once it does, is its paid execution faster? A favorable answer to either question leaves the other unresolved.

The reference question concerns both history and the next computation. Prefill, continuation, verification, and cache reuse can follow different numerical paths. Two executions with the same committed token IDs may therefore retain different numerical states; running the same verification recipe from those states can yield different decisions. Conversely, changing the current recipe can change a decision even under the same retained state. Distinguishing these cases matters when choosing what to constrain.

The cost question requires an equally careful comparison. A repair can change the target path and its prefill behavior even before drafting is enabled. Comparing the resulting speculative execution only with the original scalar decoder then combines two gains: the target-path change and speculation on that path. We measure a paid ordinary reference R, matched draft-free execution M, and the speculative candidate C. Their joint comparison distinguishes whether the whole deployment improves and what speculation adds.

Prior work establishes the underlying numerical principles. Batch-invariant inference constrains reductions (He & Thinking Machines Lab, 2025); LLM-42 verifies with fixed shapes and repairs KV state (Gond et al., 2026); StageReplay intervenes directly on cached state (Lorup, 2026); and Vosti verifies system-level determinism (Qin et al., 2026). Our focus is the intersection of reference preservation and measured cost in quantized consumer deployments. We contribute three sets of evidence:

- **State–path diagnosis.** Natural retained-state crosses on CUDA and Vulkan exhibit path effects, state effects, interactions, and selected-boundary nulls. These observations distinguish repair targets that equal token prefixes alone cannot identify.
- **A qualified positive operating point.** A finite query-layout constraint on RTX target prefill and continuation preserves complete reference arrays on 24 project-unused inputs. Paid ordinary and matched comparators both support positive generation speed.
- **Qualification and cost boundaries.** Wider fixed-capacity drafting stays qualified but becomes slower; a B570 repair improves an older speculative implementation yet loses substantially to ordinary and matched

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Evidence-frozen revision v0.12.1. Author review draft; not submitted.

decoding. The comparators expose these failures directly.

The transferable result is a sequence of tests: separate state from current execution, qualify registered outputs, and compare both draft-free baselines. Late repair evidence predominantly uses Qwen3; historical M4 and 358H costs retain different contracts.

2 Reference, State, and Cost Contracts

Fix model weights, tokenizer, prompt format, sampler, runtime, backend, and an output endpoint. Let $R(x)$ denote the paid ordinary reference for input x. Qualification checks submitted token IDs, length, terminal token when available, and stop reason. It is an output-reproduction requirement, not code correctness, equal logits, or a proof of distributional equivalence.

The RTX expansion compares the full saved response arrays through the runtime EOG or token cap. For B570, the registered endpoint is the prefix through the first runtime EOG or token 128. Extra raw suffix tokens can therefore coexist with qualified endpoints; their actual work remains included in the measured candidate cost. These contracts must not be interchanged.

At a natural boundary with committed prefix p, denote the retained state by s and the current execution recipe by e. The observed decision is

$$d = F(p, s, e). \tag{1}$$

The state includes numerical KV contents and relevant runtime state, not merely token IDs. Crossing naturally reached states s_0, s_1 with recipes e_0, e_1 produces four observations. Differences between columns within a row demonstrate a current-recipe effect for that state. Differences between rows within a column demonstrate a donor-state effect. A recipe effect present under only one donor is a local interaction. These are observations at selected boundaries, not a unique-operation attribution or a global functional decomposition of F.

For timing, let $t_{i,b}$ and $t_{i,c}$ be each task's medians of absolute baseline and candidate costs over the campaign's repetitions. We report

$$S_{b/c} = \exp\left(\frac{1}{n}\sum_{i=1}^{n}\log\frac{t_{i,b}}{t_{i,c}}\right). \tag{2}$$

Values above one favor the candidate. Repetitions are not independent tasks. Intervals resample tasks after collapsing repetitions and describe the measured panel, rather than a model-family or hardware population.

Two comparators answer different questions. Ordinary R measures the full improvement over the selected paid reference. Matched M uses the candidate target path and prefill configuration without drafting, measuring speculation's increment on that path. Candidate C combines that target path with native drafting. For the same tasks, timers, and aggregation,

$$S_{R/C} = S_{R/M}S_{M/C}. \tag{3}$$

This is an arithmetic identity, not a component-cost model. Differences in prefill or continuation cannot be converted into measured kernel fractions without profiling. Nor does a qualified R establish the fastest possible ordinary decoder.

Joint decision. For a frozen panel and endpoint, output qualification is a gate before cost: a candidate with any registered output failure has not established reference-preserving speed. Among qualified candidates, $S_{R/C}$ measures deployment improvement and $S_{M/C}$ measures speculation's increment. Figure 3 reports the task-level variation behind these aggregates. A positive geometric mean can coexist with task regressions. These gates describe the measured operating points; they do not guarantee unseen-input preservation or global optimality.

3 Experimental Sequence and Evidence Strength

The late RTX studies use an RTX 5070 Ti Laptop, Qwen3-8B/Qwen3-0.6B Q8_0, the frozen non-flash CUDA graph, one slot, greedy sampling with seed 123, context 4096, and 24 host threads. Exposure accounting before reference generation identifies 24 eligible MBPP (Austin et al., 2021) inputs. They are unused in accessible project history, rather than absent from pretraining or a new model family.

The sequence separates four exposed HumanEval (Chen et al., 2021) state probes, a 16-mask factorial, exposed qualification/cost panels, the frozen input expansion, and width/length tests. Discovery outputs are not holdouts; screening costs are not pooled with confirmation. A separate resident serial-request study tests request reset.

The RTX expansion has four conditions, 24 tasks, and four repetitions: 384 formal paid requests and 16 excluded warmups. Each condition has 96 qualified responses with stable references. Output lengths range from 11 to 128; 23 tasks end at EOG and one at the cap. Capture is off. Balanced condition-specific resident sessions measure single-request costs.

The ordinary-reference analysis reconstructs this existing campaign on CPU without new inference. Current public-table arithmetic and hash checks inherit original-host private-output and timer verification; they do not independently reopen private archives.

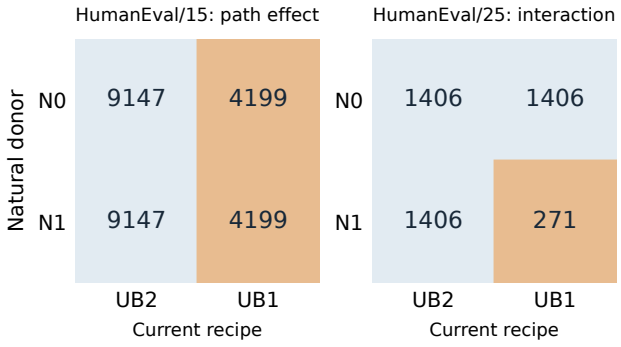


Figure 1. RTX natural-state crosses at fixed committed prefixes. /15 shows a current-path effect under both donors; /25 changes only for N1 with UB1. Cells display the discriminating sampled token ID: the third decision for /15 and the first for /25. Colors group equal IDs within each task. These selected cells are local causal observations.

B570 uses its own Qwen3/Vulkan phase-binding recipe and endpoint contract. Historical M4 and 358H data retain their own references, timers, and models. Source-specific denominators prevent repeated requests or cap contexts from inflating independent-input counts.

4 Natural State Crosses

4.1 RTX: path effects and local interactions

Four previously exposed HumanEval probes use naturally reached donor states at equal committed prefixes. Across repetitions, 24 donors generate 48 state/current-path forks with 24 own-path shams. Auxiliary probe forwards are separately charged and are not speed measurements. Restoring and continuing an own-path state checks that the manipulation itself reproduces the native decision.

At HumanEval/15, the final verification/bonus decision changes with the current UB2 versus UB1 call under both donors. At HumanEval/25, only the combination of the N1 donor and current UB1 call changes the sampled decision. The latter is an interaction, not a state-only explanation. At the chosen boundaries of /18 and /31, neither intervention changes the discrete decision. A null at these boundaries does not prove trajectory-wide invariance.

The donors' prompt K/V tensors agree in all 12 compared pairs, while 70 of 72 generated-prefix tensor comparisons differ. First-layer logical K/V remain equal in the tested pairs.

The expanded width study supplies another case. MBPP/229 at a natural 47-token prefix fails at wide UB6 verification. Real donor-state crosses under two current query layouts flip EOG argmax with the state; current layout changes continuous scores without independently flipping argmax in the measured cells. Four shams reproduce their own decisions. Two cap contexts are two observations of one task, not two independent counterexamples. This case connects a numerical-state boundary to a wider qualified-execution search without claiming a uniquely causal CUDA kernel.

Table 1. Representative B570 2×2 argmax matrices. Rows are natural donor states; columns are the recorded current recipes. /72 has duplicate columns. Cases are local evidence, not prevalence estimates.

Task	Donor 0: (e_0, e_1)	Donor 1: (e_0, e_1)
HumanEval/145	$(71465, 279)$	$(279, 279)$
HumanEval/42	$(432, 1817)$	$(1817, 1817)$
MBPP/72	$(320, 320)$	$(308, 308)$

4.2 B570: a different recipe and the same questions

On B570, six natural-state crosses with 12 own-path shams cover three tasks in repeated contexts. Byte-validated round trips and KV ownership checks support genuine donor-state manipulation. HumanEval/145 and /42 show local state–whole-recipe interactions. MBPP/72 shows a donor-state effect while its two current-recipe columns are identical controls. It cannot count as an independently varied current-path effect.

Table 1 shows representative raw argmax IDs, whose numeric magnitudes have no metric interpretation. The current B570 recipes change microbatch and output-reservation behavior together. The matrix therefore does not isolate one operator. Subsequent counterfactual complete continuations are not available for these cross cells; endpoint recovery is demonstrated separately by native full requests.

Together the two GPUs support the portability of the *experimental question*: equal prefixes do not remove the need to control retained state and the current recipe. They do not demonstrate that one constraint, matrix pattern, or parameter choice transfers unchanged between CUDA and Vulkan.

5 A Finite RTX Repair with Two Baselines

5.1 Local layout and scope

The RTX implementation changes the query layout of the Qwen3 non-flash attention subgraph using existing ggml views and CUDA operations. Query columns are represented in the batch dimension so that attention follows the selected scalar-query structure; projection and feed-forward operations retain their batched execution. This is a local graph transformation, not a new CUDA kernel or architecture-independent arithmetic proof.

Four switches apply the constraint to target prefill, target continuation, draft prefill and draft continuation, encoded by bits 1, 2, 4 and 8. The exposed factorial contains 16 masks, four probes and two repetitions, totaling 128 complete requests. Masks 3, 7, 11 and 15 pass every registered reference endpoint. Mask 3 is inclusion-minimal within this finite family. /15 and /31 pass all masks; /25 needs either target phase; /18 needs both. Draft constraints are unnecessary for qualification on this panel.

Mask 3 is chosen by output qualification, not observed speed. On 19 exposed tasks, the subsequent six-repetition panel yields 1.27808× generation and 1.11298× resident-request speed against matched target-only execution. Removing draft constraints relative to mask 15 adds only 1.00357× generation and 1.02569× request speed. The extra generation benefit misses the predeclared 1.03× gate, and its descriptive interval crosses one.

5.2 Expanded input measurements

The frozen mask-3, $k = 3$, UB4 recipe qualifies all 96 candidate responses on the 24-input MBPP expansion. The all-scope and matched-target conditions also qualify. Table 2 reports the final paid-reference reconstruction from the same campaign. The ordinary condition is mask 0, UB1, draft-free execution in the qualified frontend. It is a real measured scalar reference, not an unmeasured optimum over all possible decoding implementations.

Both generation comparators support a finite positive result. The request result requires more care. Matched draft-free execution already achieves 1.68256× against ordinary R, whereas adding speculation on that path contributes 1.03507×. The 1.74157× total gain must therefore not be described as entirely caused by speculation. Equation 3 holds for all 48 task-level generation/request identities and both aggregate identities. It apportions comparisons, not measured implementation components.

The matched-request result is modest and uneven: seven tasks are slower, and repetition 2 has aggregate speed 0.94022×. Within-task medians and balanced session order expose variability but cannot eliminate thermal drift or condition-specific session confounding on a laptop. Intervals do not turn this campaign into independent multi-day replication.

The generation timer is the original server `predicted_ms` field. It excludes prompt processing, the first sample and some final bookkeeping. Resident-request time is host HTTP response wall time, including prefill, generation, transport and client parsing. It excludes loading, warmup and shutdown and may exclude asynchronous post-response cleanup. The R versus M comparison changes prefill microbatch/attention behavior as well as continuation capacity.

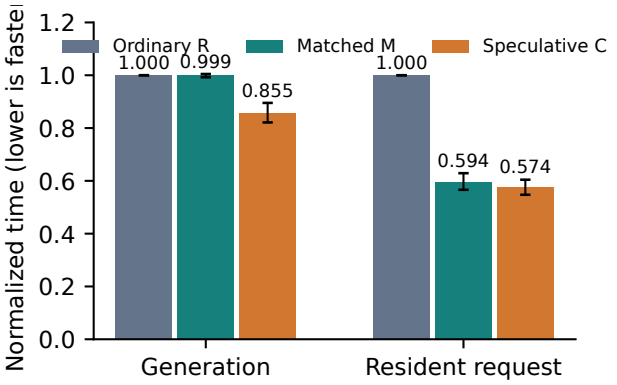


Figure 2. RTX expansion costs normalized task-by-task to ordinary R, then geometrically averaged. Matched draft-free execution already reduces request cost substantially. Error bars invert the descriptive task-bootstrap intervals for R/M and R/C. Bars are aggregate relative times, not additive cost components.

No per-component timing establishes where its large request gain is spent. The target-state transplant branch is disabled in this campaign; code availability is not evidence that transfer cost was paid here.

6 Qualification and Amortization Boundaries

Table 3 summarizes the distinct outcomes. Passing the output gate does not imply passing the cost gate, and a failed output gate disqualifies a speed claim even if the recorded run is fast. The rows come from separate registered panels; they are not a pooled benchmark.

6.1 Changing geometry can break the repair

The expansion into draft length and microbatch capacity keeps failed cases. At paired $(k, \mathrm{UB}) = (1, 2)$ and $(3, 4)$, the tested width/length panel qualifies. MBPP/229 fails at $(5, 6)$ and $(7, 8)$ under all measured cap contexts and repetitions, even though its wide ordinary and matched paths pass. The old HumanEval/18 probe also supplies an outside-panel failure at wider target-only capacity. Increasing the output cap does not certify every trajectory to that length: some responses finish much earlier, with observed maximum length 261 rather than a full 512-token stress trajectory.

A separate experiment holds UB4 fixed and tests $k = 3, 5, 7$. All 28 exposed qualification tasks pass in two repetitions. The widest $k = 7$ recipe is selected by qualification only, then measured in independent capture-off cost sessions. Against $k = 3$, it obtains 0.82944× generation and 0.88987× request speed: qualified outputs, slower execution. Against matched target-only, $k = 7$ reaches 0.99740×

Table 2. RTX expansion: 24 tasks, four repetitions per condition. Baseline/candidate geometric mean of task-median absolute times; above one favors the candidate. Brackets are descriptive task-bootstrap 95% intervals. Ordinary R and matched M are both draft-free.

Comparison	Generation speed	Resident-request speed	Generation faster	Request faster
R/C: ordinary / mask-3 speculation	1.17012 [1.11701, 1.21768]	1.74157 [1.65461, 1.82708]	22/24	24/24
M/C: matched / mask-3 speculation	1.16924 [1.11711, 1.21715]	1.03507 [1.00455, 1.06348]	22/24	17/24
R/M: ordinary / matched draft-free	1.00076 [0.99529, 1.00775]	1.68256 [1.58980, 1.76633]	11/24	24/24

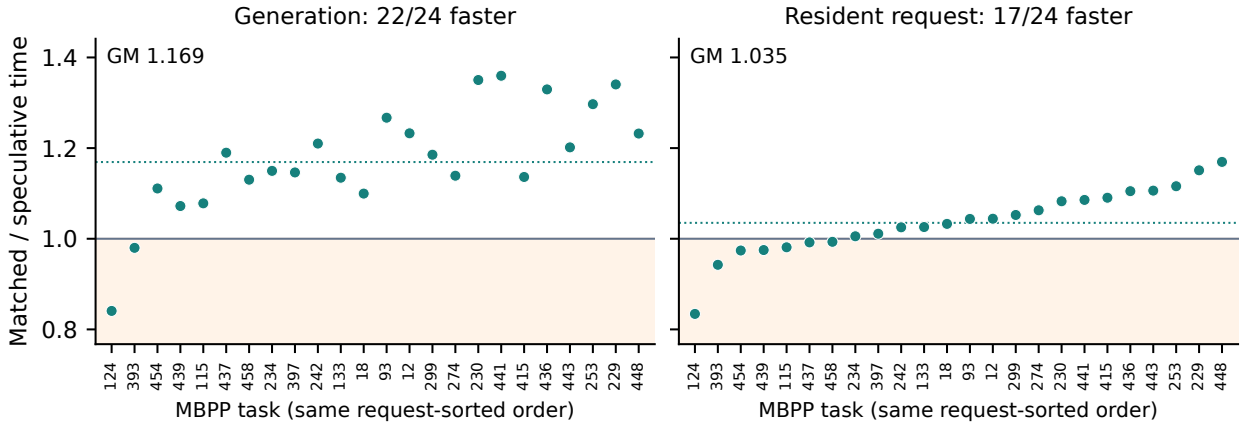


Figure 3. RTX expansion: each point is the matched-target/candidate ratio of four-repetition absolute-time medians for one MBPP task. Both panels use the same order, sorted by request ratio. Generation improves on 22/24 tasks; resident-request time improves on 17/24. The line at one marks parity; the horizontal dotted line is the unweighted task geometric mean. Points are tasks, not independent repetitions.

Table 3. Observed qualification/cost combinations. Gen./req. speeds are against the named draft-free baseline; – means no qualified speed claim.

Recipe	Qualified panel	Baseline	Gen./req.
RTX $k = 3$, UB4	24 tasks	Matched	1.169/1.035
RTX $k = 7$, UB4	28 tasks	Matched	0.997/0.950
RTX $k = 7$, UB8	Fails /229	–	–
B570 $k = 2$ repair	15 tasks	Ordinary	0.283/0.201

generation and 0.95035× request speed; the $k = 3$ anchor reaches 1.20250× and 1.06797×. Larger k is neither a monotone correctness repair nor a monotone performance improvement in these contracts.

Diagnostic sessions, including warmup, record target caller calls falling from 305 to 199 while observed micrographs rise from 305 to 397. Separate formal work records show drafted tokens increasing from 3528 to 5392 and accepted tokens from 2772 to 3192. These are different sessions and must not be divided into one fabricated per-call measure. Micrograph observations are not CUDA kernel launches. The counts reveal why caller reduction is an inadequate performance proxy; they do not uniquely prove which component caused the slowdown. This limitation is consistent with prior work on verification cost (Liu et al., 2026b), while the fixed-capacity, reference-qualified counterexample adds a concrete numerical-contract boundary.

The resident serial-request study verifies reset behavior and positive finite costs on its own panel, approximately 1.181× generation and 1.039× request against its matched comparator. It is not a throughput or concurrency benchmark and is not pooled with the expansion.

6.2 B570: repair without deployment gain

The B570 frontend combines ordinary-reference prompt construction, in-place prefill-to-continuation phase binding from UB128 to UB1, and the existing $k = 2$, draft-UB1 speculative loop. It qualifies 12 development requests, 112 requests over 56 exposed tasks, and all 30 candidate requests over 15 inputs unused in the accessible project. Qualification is the cap-128 endpoint contract from Section 2. The 30 candidate responses include extra raw suffix differences; endpoint equality cannot be rewritten as raw-array equality.

The formal E matrix is 119/120 complete because one old-N1 request lacks a payload. Ordinary R, matched target-only, and the candidate each have all 30 paid observations, so their 15 task-median pairs remain complete. Candidate speed is 0.28259× generation and 0.20129× request against ordinary R, slower in every paired round. Against matched target-only it is 0.28576× and 0.20570×. Matched target-only itself is near the ordinary reference: 0.98889× and 0.97854×. Thus the entire candidate regression cannot be attributed to a target consistency constraint alone.

Against 14 complete old-N1 task pairs, the candidate improves generation/request by 3.99309×/2.63394×, with a single request-round reversal at MBPP/245. This old-implementation improvement does not establish a deployable gain. Draft, accepted, caller and extra-suffix work are recorded, but no independent component profile identifies a unique remaining bottleneck. Further GPU investment in this exact route is not supported by these measurements.

6.3 Historical opportunities on M4 and 358H

Historical M4 data supply a positive cost case. A prospective 60-task panel, 30 HumanEval plus 30 SpecBench (Xia et al., 2024) inputs, has three conditions and three repetitions: 540 requests. Fixed Always speculation and Guard each qualify 180/180 responses under that study's contract. Their post-prefill speedups against target-only are 1.34689× and 1.31139×. Guard is 2.71% slower than Always in geometric task latency. This guard has no measured advantage on the prospective panel.

A whole-request offline oracle choosing between target-only and Always by each observed task's medians improves on Always by only about 0.0692% in geometric mean. It is a panel-specific choice oracle, not an upper bound for intra-request policies or other implementations. On Core Ultra 358H, CPU and NPU drafting reproduce the reference on their 90-request panels but yield 0.73392× and 0.35166× speed. Conditional removal of measured overhead leaves the available ceilings below parity. Full-prefix NPU replay and the historical target path are properties of that implementation, not impossibility results for heterogeneous decoding.

7 Related Work

Speculative sampling (Leviathan et al., 2023), heterogeneous drafting (Zhang et al., 2025; Lv et al., 2025), and adaptive depth (Liu et al., 2025; Hou et al., 2025) are established directions. Consumer numerical deviations and serialization costs are also documented (Chordiya, 2026). Our study asks which concrete numerical repairs qualify and repay their cost.

LLM-42 combines fixed-shape verification, consistent KV state, and selective deterministic requests (Gond et al., 2026). Those principles overlap our constrained target path and unconstrained draft scope. Vosti specifies and verifies system-level determinism (Qin et al., 2026), providing a stronger guarantee than finite output qualification. MarginGate uses calibrated sparse verification (Chu et al., 2026). We have no shared-contract implementation comparison with these methods.

Stage-Replay holds prefixes fixed and uses bidirectional cache transplantation (Lorup, 2026). Our additional measured factor is the current native recipe crossed with natural retained state, linked to repair scope and paid cost. State intervention itself has precedent. *Performance or Illusion?* studies verification costs and performance bounds in a production engine (Liu et al., 2026b); our fixed-capacity experiments add a reference-qualified numerical boundary and ordinary/matched comparisons.

An ACL paper also uses the name HeteroSpec for uncertainty-based verification allocation (Liu et al., 2026a). We retain a distinct descriptive manuscript title and cite that work.

8 Limits and Implications

The late mechanism and repair evidence predominantly uses Qwen3, greedy decoding, one slot, and the recorded non-flash paths. CUDA and Vulkan use different recipes; M4 and 358H contribute historical cost cases. Selected-state crosses establish local effects with finite shams, rather than prevalence or a unique kernel cause. Scope minimality is limited to the tested switches. Token agreement is weaker than equal logits or equal retained state; B570 endpoint agreement also differs from raw-array equality.

The RTX ordinary reference is paid and qualified, but is not established as the fastest ordinary decoder. Balanced resident sessions and task bootstrap expose variation without removing session confounding. Public-table checking inherits its original-host private-archive verification. Cross-family, long-context, stochastic, concurrent, and nearest-method comparisons remain open.

The practical implication is to diagnose state and current execution separately, qualify registered outputs, and compare with both ordinary and matched draft-free execution. The observed operating points show why every step changes the interpretation of a speed claim.

9 Conclusion

Reference-preserving speculation is useful where numerical qualification and amortization hold together. Natural state–current-path crosses distinguish local failure patterns that equal token prefixes hide. On RTX, a finite target-layout constraint reaches a qualified operating point with positive generation performance against both draft-free baselines. The matched comparison also shows that much of its ordinary-reference request gain is already available without speculation. Wider fixed-capacity drafting and B570 endpoint repair provide qualified cost counterexamples. These cases motivate joint output and two-baseline evaluation, while leaving cross-family repair and broad deployment transfer open.

REFERENCES

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., and Sutton, C. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Ponde de Oliveira Pinto, H., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Petroski Such, F., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Hebgen Guss, W., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374v2*, 2021.

Chordiya, P. Lossless but not free: An empirical anatomy of speculative decoding on consumer hardware. *arXiv preprint arXiv:2607.17283*, 2026. doi: 10.48550/arXiv.2607.17283.

Chu, K., Zhou, Y., and Zhang, W. MarginGate: Sparse margin-triggered verification for batch-invariant LLM inference. *arXiv preprint arXiv:2605.30218v1*, 2026.

Gond, R., Kamath, A. K., Ramjee, R., and Panwar, A. LLM-42: Enabling determinism in LLM inference with verified speculation. *arXiv preprint arXiv:2601.17768v2*, 2026.

He, H. and Thinking Machines Lab. Defeating nondeterminism in llm inference. *Thinking Machines Lab: Connectionism*, 2025. doi: 10.64434/tml.20250910. URL https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/.

Hou, Y., Zhang, F., Du, C., Zhang, X., Pan, J., Pang, T., Du, C., Tan, V., and Yang, Z. BanditSpec: Adaptive speculative decoding via bandit algorithms. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pp. 24045–24079. PMLR, 2025. URL https://proceedings.mlr.press/v267/hou25h.html.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pp. 19274–19286. PMLR, 2023. URL https://proceedings.mlr.press/v202/leviathan23a.html.

Liu, S., Ye, Y., Zhu, Q., Cao, Z., and He, Y. HeteroSpec: Leveraging contextual heterogeneity for efficient speculative decoding. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 12930–12947. Association for Computational Linguistics, July 2026a. doi: 10.18653/v1/2026.acl-long.589. URL https://aclanthology.org/2026.acl-long.589/.

Liu, X., Park, J., Hu, L., Kwon, W., Li, Z., Zhang, C., Du, K., Mo, X., You, K., Cheung, A., Deng, Z., Stoica, I., and Zhang, H. TurboSpec: Closed-loop speculation control system for optimizing LLM serving goodput. *arXiv preprint arXiv:2406.14066v3*, 2025.

Liu, X., Yu, J., Park, J., Stoica, I., and Cheung, A. Speculative decoding: Performance or illusion? In *Proceedings of Machine Learning and Systems*, volume 8, pp. 1719–1741, 2026b. URL https://proceedings.mlsys.org/paper_files/paper/2026/hash/554e056fe2b6d9fd27ffcd3367ae1267-Abstract-Conference.html.

Lorup, A. B. Stage-replay divergence follows the KV cache: Fixed-prefix precision controls and bidirectional cache transplantation. *arXiv preprint arXiv:2607.28495v1*, 2026.

Lv, K., Guo, H., Guo, Q., and Qiu, X. DuoDecoding: Hardware-aware heterogeneous speculative decoding with dynamic multi-sequence drafting. *arXiv preprint arXiv:2503.00784*, 2025.

Qin, J., Du, A., Zhang, D., Lentz, M., and Zhuo, D. Vosti: Specifying, implementing, and verifying deterministic llm inference. *arXiv preprint arXiv:2609.38981v1*, 2026.

Xia, H., Yang, Z., Dong, Q., Wang, P., Li, Y., Ge, T., Liu, T., Li, W., and Sui, Z. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 7655–7671. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-acl.456. URL https://aclanthology.org/2024.findings-acl.456/.

Zhang, L., Zhang, Z., Xu, B., Li, R., Tian, Z., Mei, S., and Li, D. Dovetail: A CPU/GPU heterogeneous speculative decoding for LLM inference. *arXiv preprint arXiv:2412.18934v2*, 2025.